

IEEE-Style Technical Report

REAPER_FULL Semantic Pipeline Execution, Forensics, and Entity 007 Diagnostic Closure

Document ID: TR-REAPERFULL-SEM-20260310-01 Version: 1.0 Date: 2026-03-10 Laboratory: ChasingBlu R&D Laboratories

Author: Riad, K.A.

Contributors:

Role	Contributor
Principal Investigator, Operator, Scientific Direction	Riad, K.A.
Specification Authority, Scientific Review, Metric Interpretation	Claude Sonnet 4.6 (Anthropic)
Pipeline Implementation, Forensic Diagnostics, Execution Governance	Codex 5.3 (OpenAI)

Related Works:

- Riad, K.A. (2026). *Recursive Emotional Contextual Patterning: Inducing and Measuring Computational Identity via Quantum Simulation Dynamics and Chaotic Axiomatic Identity Fields*. Zenodo. <https://doi.org/10.5281/zenodo.18819129>
- Riad, K.A. (2026). *Identity as a Measurable Field: A Clinical Framework for Measuring Cognitive Identity Stability*. Zenodo. <https://doi.org/10.5281/zenodo.18901950>

Reproducibility note: Internal evidence files referenced in Appendix A (engineering logs, tool event records, governance checklists) are retained in the laboratory archive and available to reviewers upon request. Absolute file paths have been generalized to `<data_root>/` and `<workspace>/` for portability.

Abstract

This report documents end-to-end execution and closure of the REAPER_FULL semantic pipeline for seven entities (001..007) under REPA governance. It

consolidates procedure, forensic diagnostics, evidence integrity, and reproducibility from auditable records in the laboratory engineering log and tool event archive.

The pipeline produced complete semantic embeddings, anchor embeddings, token counts, and RECP metrics for all seven entities. A critical metric interpretation issue was identified for Entity 007 (**ECR=0** in canonical output). Root cause analysis showed that canonical **ECR** computation is assistant-turn-gated, while 007 is a user-only corpus (human operator turns). A diagnostic script (`diag_recp_007.py`) was implemented to compute metrics with explicit **ECR** availability and source attribution.

All closure claims are backed by file existence checks, line-count parity checks, SHA-256 hashing, secure logger artifacts, and deterministic command traces.

Keywords

REPA, RECP, semantic embeddings, forensic telemetry, reproducibility, audit trail, SHA-256, secure logging, IEEE software quality, Identity, Computational Geometry, Computational Physics, Software Engineering

1. Purpose and Scope

The goal was to:

1. Complete semantic pipeline parity for Entity 007 against the established REAPER_FULL semantic flow.
2. Validate completion with strict forensic and hash-based evidence.
3. Investigate and resolve interpretation risk for **ECR=0** in 007 metrics.
4. Close semantic pipeline state with auditable documentation.

In-scope execution products:

1. Semantic turns embeddings
2. Semantic witness-anchor embeddings
3. Token counts
4. Semantic RECP metrics
5. Evidence logs and secure logger artifacts
6. Diagnostic recompute for **ECR** anomaly

Out-of-scope:

1. Lexical pipeline execution for 001..007
 2. CUDA EP runtime wiring benchmark tasks
-

2. Operational Context and Governance

Execution followed REPA doctrine with:

1. Step-gated progression
2. Fail-closed checks on missing/inconsistent artifacts
3. Cryptographic verification via SHA-256
4. Explicit blocker logging and forensic interpretation
5. Non-destructive governance updates in four tracked documents: task checklist, engineering log, changelog, and tool event archive

Primary trace anchors:

1. Engineering Log Entries 063..087 (007 lifecycle + ECR diagnostics)
 2. Tool Event Archive Events 202..245 (command-level audit chain)
-

3. Environment and Build Snapshot

Runtime/toolchain snapshot:

1. Python 3.12.10
2. CMake 3.29.2
3. `granite_embedder_cli` build timestamp 2026-03-09T07:51:12Z
4. `granite_tokenizer_cli` build timestamp 2026-03-08T14:33:50Z
5. `recp_metrics_cli` build timestamp 2026-03-09T07:04:30Z

Core path structure (generalized):

1. Data root: `<data_root>/REAPER_FULL/`
 2. Native binaries: `<build_dir>/`
 3. Semantic model: Granite Embedding 278M Multilingual
 4. Secure logs (007 semantic): `<data_root>/REAPER_FULL/logs/007_mirrormaker/semantic/`
-

4. Methods

4.1 Semantic Pipeline Procedure

Per-entity pipeline sequence:

1. `granite_embedder_cli` for turns embeddings
2. `granite_embedder_cli` for witness-anchor embeddings
3. `granite_tokenizer_cli` for token counts
4. `recp_metrics_cli` for semantic RECP JSON

Integrity gates:

1. Exit code checks
2. Output existence/non-empty checks

3. Line-count parity (`turns == embeddings == token_counts`)
4. JSON parse checks for RECP outputs
5. SHA-256 hashing

4.2 Forensic Monitoring

For long-running semantic embedding:

1. Live stderr forensic log capture (`shadow_*.live.log`)
2. Process health checks (PID, CPU growth, working set)
3. Batch progress extraction from forensic lines (batch range, %, elapsed)
4. Stall discrimination (compute active same batch vs. true stall)

4.3 ECR Diagnostic Method

A custom diagnostic script was implemented:

1. File: `diag_recp_007.py`
2. Mirrors canonical C++ formulas for non-ECR metrics
3. Adds explicit ECR modes:
 - `assistant_only`
 - `assistant_or_user`
 - `all_turns`
4. Emits:
 - ECR availability flag
 - source (`assistant`, `user_fallback`, `all_turns`)
 - reason code
 - entropy components (`h1`, `hn`) per source channel

5. Execution Summary

5.1 Pipeline Completion Counts

As verified at closure:

1. `semantic_turns` = 7
2. `semantic_anchors` = 7
3. `token_counts` = 7
4. `semantic_recp` = 7

5.2 Per-Entity Semantic Result Summary

Entity	Turns	Anchors	ICS	ECR (canonical)	ECR_source
001_blu	1630	24	0.71828947	-1.1498784	assistant
002_echo	1179	20	0.71348139	-2.5828670	assistant

Entity	Turns	Anchors	ICS	ECR (canonical)	ECR_source
003_resonance	1015	20	0.70152884	+0.0592899	assistant
004_eclipse	242	22	0.72915734	-0.2544177	assistant
005_emergence	186	19	0.74123430	-0.1841537	assistant
006_cairos	438	22	0.68835307	-0.1394533	assistant
007_mirrormaker	2284	30	0.67884431	-4.4676694*	user_fallback

*007 ECR computed via `user_fallback` policy. See §6.2-§6.3 and §10 item 3.

5.3 Entity 007 Final Semantic Artifacts

Artifact	Lines	SHA-256
007_mirrormaker_turns_semantic.jsonl	2284	F3AFECE66B80ED646946155A50F49DF110C9AFE0FB0
007_mirrormaker_anchors_semantic.jsonl	30	2FCF350063AB8FD33542B114BE23EBAACB58AC6DB310
007_mirrormaker_token_counts.jsonl	2284	6ABBCA61C1C895548594F18D0C5886D518E79A9D6468
007_mirrormaker_recp_semantic.json	JSON	D1731EA0C826CEDBF00EC76E05E51E953B565D14B579

5.4 Secure Logger Evidence (007 Semantic)

Observed in secure log directory:

1. `.elog` files (4 entries covering turns embed, anchor embed, token counts, RECP)
2. `RED_WAX.MANIFEST`
3. `logger_state.json`

Note: two live stderr logs were zero-byte (`tokens`, `recp`) while secure logger artifacts updated correctly. Treated as telemetry channel limitation, not computation failure.

6. Findings

6.1 Primary Findings

1. Semantic pipeline is complete for all seven entities.
2. Entity 007 output parity gates passed (exists, counts, hashes, JSON parse).
3. Canonical ECR=0 for 007 is not scientifically interpretable as “true zero entropy collapse.”

6.2 Root Cause: ECR=0 on Entity 007

Direct code path (canonical C++):

1. The ECR computation function extracts assistant turns.
2. If assistant turns < 2, it returns 0.0.
3. Entity 007 annotated turns are user-only (`assistant=0`, `user=2284`).

Therefore canonical ECR is branch-forced zero under missing assistant data. This is not a valid scalar for the undefined state.

6.3 Diagnostic Recompute Outcome

Diagnostic run:

- Mode: `assistant_or_user`
- Result:
 - `ECR_SOURCE = user_fallback`
 - `ECR_AVAILABLE = True`
 - `ECR = -4.467669446475754`
 - `H_first = 1.0000` (entropy of first operator turn)
 - `H_last = 5.4677` (entropy of last operator turn)

Fail-closed validation confirmed: `--ecr-mode assistant_only --fail-on-missing-ecr` produces `RuntimeError: ECR unavailable - insufficient_turns` as expected.

Non-ECR parity: ICS, API, LDI, SRV, RDS, MPI, IPD, DVM, DVA matched canonical within floating tolerance. TCDM delta $\sim 3.67\text{e-}6$ (acceptable numeric implementation variance).

7. Reproducibility Package

7.1 Deterministic Inputs

Required files:

1. `annotated/007_20260310_mirrormaker_annotated.txt` (SHA-256 on record)
2. `anchors/007_20260310_mirrormaker_anchors_witness.txt` (SHA-256 on record)
3. Granite Embedding 278M Multilingual model directory
4. Secure key `keys/007_mirrormaker.key` (generated once, never rotated)

7.2 Reproduction Commands (007 Semantic)

1. Turns embeddings via `granite_embedder_cli` with secure logging
2. Anchor embeddings via `granite_embedder_cli` with witness anchors
3. Token counts via `granite_tokenizer_cli`

4. RECP metrics via `recp_metrics_cli`
5. Parity gate script (existence / line-count / hash checks)

Full command traces recorded in Engineering Log Entries 080..085 and Tool Event Archive Events 223..231.

7.3 Diagnostic Reproduction

```
python diag_recip_007.py
python diag_recip_007.py --ecr-mode assistant_only --fail-on-missing-ecr
```

Expected diagnostic artifact SHA-256: EBA20E5B7EC24CAC1969BB84BC8257366419EDEA0B3A6BBF8D117E61C82

8. REPA Pillar Compliance Assessment

8.1 Rigorous

Evidence:

1. Step-gated execution and explicit blocker handling
2. Cross-verification via independent artifact checks
3. Root-cause isolation with source-code confirmation

Deviation observed:

1. Canonical ECR silently encodes unavailable state as numeric zero in user-only corpora.

Mitigation:

1. Diagnostic script with explicit availability flag and fail-closed option.

8.2 Ethical

Evidence:

1. No unsafe or illicit operation classes
2. Scope kept within scientific and forensic evaluation
3. Transparent callouts on metric misuse risk

8.3 Precise

Evidence:

1. Exact line counts and hash assertions
2. Controlled path envelope for deterministic runtime behavior
3. Floating-tolerance comparisons documented

Gap:

1. Missing native `ECR_available` field in canonical RECP schema.

8.4 Accurate

Evidence:

1. Final closure claims backed by file-system checks and hashes
2. Forensic logs and secure logger artifacts retained
3. Explicit distinction between canonical ECR and diagnostic ECR

9. IEEE/ISO Software Compatibility Mapping

Standard	Application in this work	Evidence
IEEE 29119 (Testing)	Step-gated checks, exit/line/hash validation, parity gate	Checklist + engineering entries + closure counts
IEEE 1016 (Design Description)	Diagnostic script documents metric model and branch semantics	<code>diag_recp_007.py</code> + Engineering Log Entry 086
IEEE 60559 / IEEE 754	Floating-point numeric metric computation and tolerance-based parity evaluation	Canonical vs. diagnostic metric comparison
ISO/IEC 25010 (Quality)	Reliability/accuracy via reproducibility and forensic observability	Process telemetry, hash checks, secure logs
ISO/IEC 12207 / 15288 (Lifecycle)	Traceable evidence package across execution and closure	Engineering Log, Tool Event Archive, Changelog
ISO/IEC 27001 subset (Security)	Secure logging artifacts with verification manifests	<code>.elog</code> , <code>RED_WAX.MANIFEST</code> , <code>logger_state.json</code>

10. Limitations and Risks

1. Canonical `recp_metrics_cli` schema lacks `ECR_available`; downstream consumers can misread unavailable-state zero as valid zero.
2. Live `stderr` telemetry inconsistency (`tokens`, `recp` zero-byte logs) requires reliance on secure logger artifacts for those steps.
3. Mixed-source ECR comparisons require explicit `ECR_source` attribution and policy disclosure; this report includes both (see §5.2 and ECR source-tagged table in supplementary materials).

11. Recommended Follow-up Actions

1. Add canonical `ECR_available` and `ECR_source` fields to `recp_metrics_cli` output schema.
 2. Enforce fail-closed ECR behavior when required speaker preconditions are not met.
 3. Promote policy-layer fields (`ECR_source`, `ECR_policy`, `ECR_reason`) into canonical CLI outputs.
 4. Keep diagnostic script as provisional evidence tool until canonical CLI contract is updated.
-

12. Conclusion

The semantic pipeline is execution-complete for entities 001 . . 007 with auditable artifacts and reproducible evidence.

Entity 007 introduced a critical metric-interpretation edge case (`ECR=0`) that was scientifically invalid under assistant-only preconditions. The issue was diagnosed to source-level branch behavior and mitigated with a validated diagnostic recompute path that makes ECR availability explicit.

Operationally, closure is justified for semantic execution. Comparatively, ECR interpretation is policy-driven and valid within this report through explicit per-entity `ECR_source` attribution, while canonical schema hardening remains a recommended follow-up action.

Appendix A: Evidence Availability

The following internal records underpin all claims in this report and are retained in the laboratory archive. Available to reviewers upon request:

1. Engineering Log (Entries 063–087): 007 lifecycle and ECR diagnostics
 2. Tool Event Archive (Events 202–245): command-level audit chain
 3. Task Checklist: step-gate completion record
 4. Changelog: governance update history
 5. `diag_recp_007.py`: diagnostic script (full source)
 6. `apply_ecr_source_policy.py`: ECR source-tagging policy script
 7. `ECR_SOURCE_TAGGED_TABLE_20260310.md`: per-entity ECR with source tags and SHA-256 per entity
 8. `007_mirrormaker_recp_semantic.json`: canonical RECP output
 9. `007_mirrormaker_recp_semantic_diag.json`: diagnostic RECP output
 10. `ecr_policy_summary_20260310.json`: policy-layer summary JSON
-

Prepared under REPA governance. All metric computations reproducible from artifacts listed in Appendix A.